

Integrating the Acrome Smart Motion Car with Artificial Intelligence: Challenges and Solutions

The Acrome Smart Motion Car is somewhat similar to a remote-controlled vehicle. Its working principle is based on the SMD boards inside it, which enable the motors to rotate. These motors have four operation modes, but the mode we use in this project is the Velocity mode.

The main objective of this project is as follows: to integrate the Acrome Smart Motion Kit with artificial intelligence and make it operational. For instance, there are some pre-written codes for the Smart Motion Kit, such as the "move forward" function. The AI processes the given command and sequentially calls the appropriate functions accordingly. For example, if the command is "move forward 50 cm and then turn right," the car will first move forward 50 cm, then turn right, and finally stop at the designated position. These operations were already part of the Acrome AI Project folder under the Motion Kit (Car) Control section.

However, the main issue here was that the Raspberry Pi couldn't automatically switch between Wi-Fi networks. Additionally, even if it did switch, the Raspberry Pi's IP address would change, causing the device (mobile phone) to lose communication with it.

APPLICATION STEPS:

- 1.** To use the application, you need an Android phone. The mobile app has been specifically developed for Android using Flutter. Below are the steps to set it up:
 - a.** In the repository, I've provided a link where you can directly download the application. If you prefer not to build the app yourself, simply use the link provided at the bottom of the document or in the README section to install the app.
 - b.** If you want to build the app manually, download the mobile_control folder from the repository. Open the folder in VS Code (or your preferred IDE) after ensuring Flutter is installed on your system. Run the following command to build the APK: **flutter build apk --split-per-abi**
 - c.** If you wish, you can modify the source code to adjust the design according to your preferences. Simply make changes to the files in the mobile_control folder before building the APK.

2. You need to have a Raspberry Pi device to run the application.
 - a. You should upload appfinal.py file to the Raspberry Pi. (I recommend using MobaXterm due to its ease of use.)
 - b. When flashing the Raspberry Pi image, make sure the Wi-Fi option is enabled. If Wi-Fi is not enabled during the setup process, you may need to manually configure the Wi-Fi settings later, which could complicate the setup.

STEPS BY STEPS

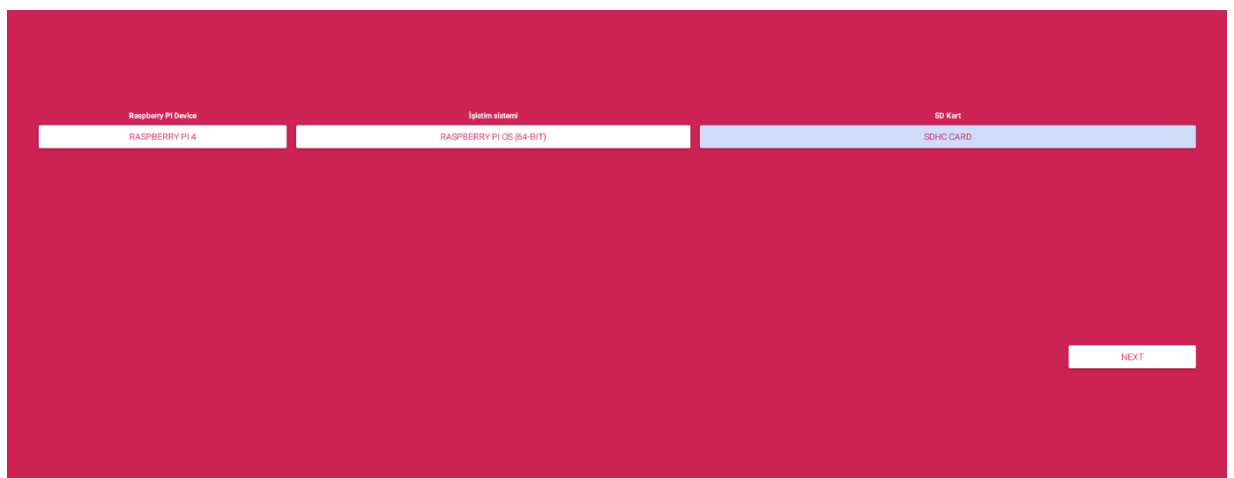
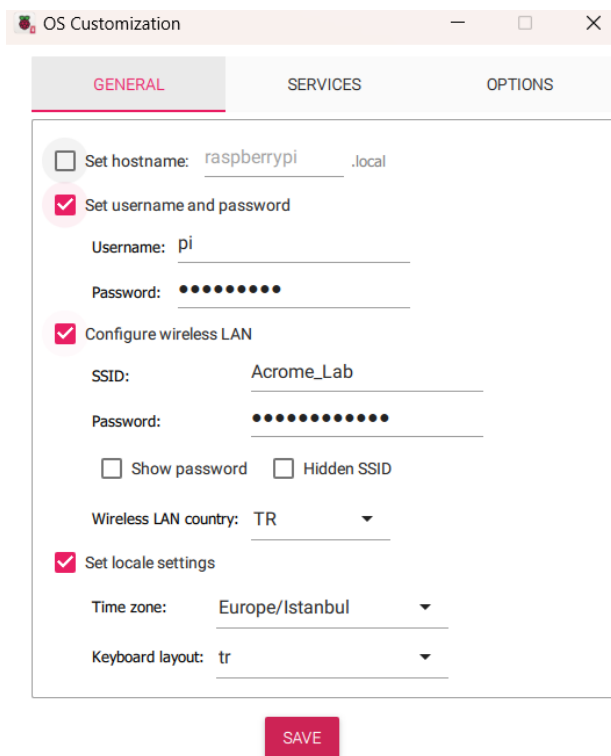


Figure 1: Raspberry pi Imager

Since this project has been tested on a Raspberry Pi 4, it is recommended to use a Raspberry Pi 4 for the best experience. Additionally, select Raspberry Pi OS (64-bit) as the operating system.

After clicking the Next button, enter the general information in the OS Customization section according to your preferences. For example:



The screenshot shows a window titled "OS Customization" with three tabs: "GENERAL", "SERVICES", and "OPTIONS". The "GENERAL" tab is active. It contains several configuration options:

- ☐ Set hostname: .local
- ☒ Set username and password
 - Username:
 - Password:
- ☒ Configure wireless LAN
 - SSID:
 - Password:
 - ☐ Show password ☐ Hidden SSID
 - Wireless LAN country:
- ☒ Set locale settings
 - Time zone:
 - Keyboard layout:

At the bottom of the window is a red "SAVE" button.

Figure 2: OS Customization.

Note: Please ensure that Configure Wireless LAN is enabled and enter your modem's SSID and password.

Then You should download this app on your phone :

https://dosya.co/ksilrqgcpak0/mobile_robot_control_global.apk.html

To find the IP address of your Raspberry Pi while it's running, follow these steps:

Install the application you downloaded.

After the installation, open the application.

The IP scanner screen will appear, displaying the available devices on your network.

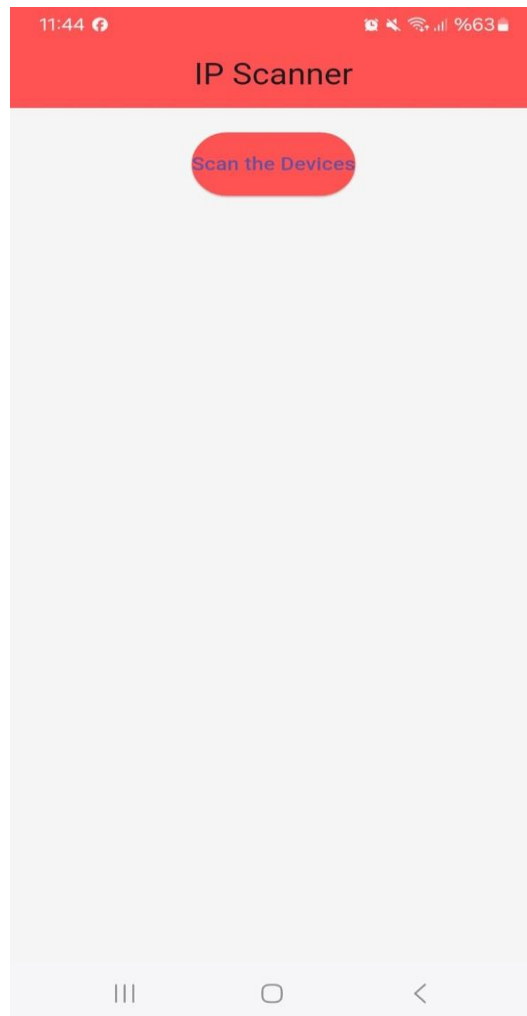


Figure3: IP Scanner

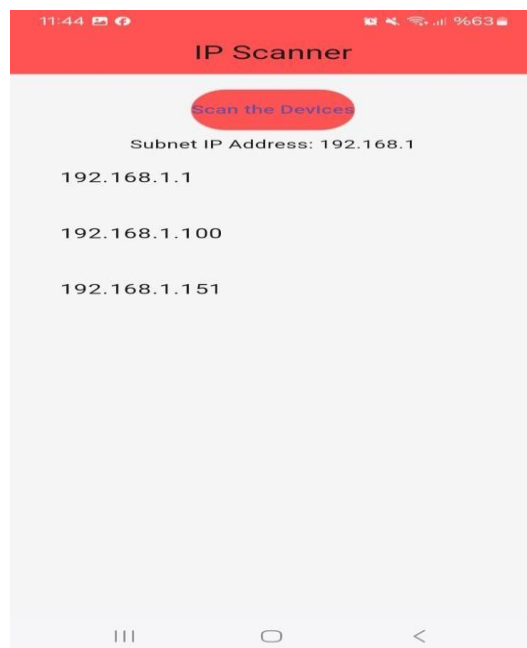


Figure4: Devices (.1 is a modem and

.100 is a router. That's why I chose 192.168.1.151 . Also this scanner just find Linux devices.)

Then, I connected to the Raspberry Pi 4 using MobaXterm.

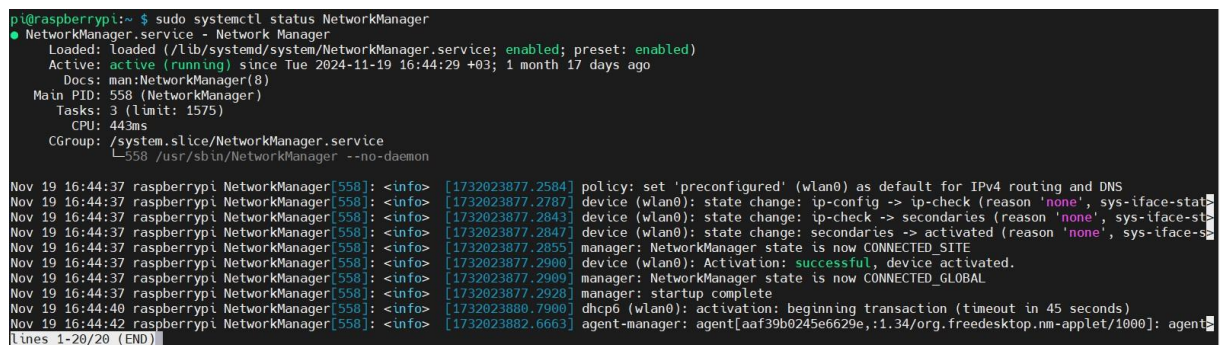
First step: You have to upload `appfinal.py` and `requirements.txt` files. Then you must write to Bash this command:

`pip install -r requirements.txt --break-system-packages`

After that you should write this command:

`sudo systemctl status NetworkManager`

And you should see like this:



```
pi@raspberrypi:~$ sudo systemctl status NetworkManager
● NetworkManager.service - Network Manager
   Loaded: loaded (/lib/systemd/system/NetworkManager.service; enabled; preset: enabled)
   Active: active (running) since Tue 2024-11-19 16:44:29 +03; 1 month 17 days ago
     Docs: man:NetworkManager(8)
    Main PID: 558 (NetworkManager)
      Tasks: 3 (Limit: 1575)
        CPU: 443ms
    CGroup: /system.slice/NetworkManager.service
            └─558 /usr/sbin/NetworkManager --no-daemon

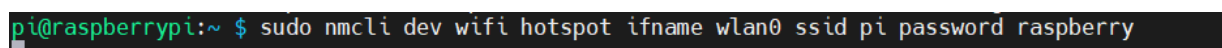
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2584] policy: set 'preconfigured' (wlan0) as default for IPv4 routing and DNS
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2787] device (wlan0): state change: ip-config -> ip-check (reason 'none', sys-iface-state='none')
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2843] device (wlan0): state change: ip-check -> secondaries (reason 'none', sys-iface-state='none')
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2847] device (wlan0): state change: secondaries -> activated (reason 'none', sys-iface-state='none')
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2855] manager: NetworkManager state is now CONNECTED_SITE
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2900] device (wlan0): Activation: successful, device activated.
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2909] manager: NetworkManager state is now CONNECTED_GLOBAL
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2928] manager: startup complete
Nov 19 16:44:40 raspberrypi NetworkManager[558]: <info> [1732023880.7900] dhcp6 (wlan0): activation: beginning transaction (timeout in 45 seconds)
Nov 19 16:44:42 raspberrypi NetworkManager[558]: <info> [1732023882.6663] agent-manager: agent[aaf39b0245e6629e,1.34/org.freedesktop.nm-applet/1000]: agent[1]
lines 1-20/20 (END)
```

Figure 5: Command Shell

After these steps, if you are going to use Raspberry somewhere else (on another network), write this line:

`sudo nmcli dev wifi hotspot ifname wlan0 ssid "Hotspot_name" password "Password"`

Like this:



```
pi@raspberrypi:~$ sudo nmcli dev wifi hotspot ifname wlan0 ssid pi password raspberry
```

Figure 6: Create Hotspot

After this, when you run **`sudo nmcli connection show`**, you will see both "preconfigured" and "Hotspot" instead of just "preconfigured" and "ethernet".

By default, **preconfigured** has auto connect set to yes. This means the Raspberry Pi will automatically connect to the **preconfigured** network if the SSID (Wi-Fi name) and password are correct and the network is available.

However, if you move to a new location or the network changes, the Raspberry Pi won't find the **preconfigured** network. To handle this, you need to run this command on the Raspberry Pi:

`sudo nmcli connection modify Hotspot connection.autoconnect yes`

This ensures that if the Raspberry Pi cannot connect to the preconfigured network, it will automatically activate the Hotspot connection.

But how does the system decide between connecting to preconfigured Wi-Fi and starting the Hotspot? The Raspberry Pi prioritizes based on the last active connection before shutdown. For example:

- If the system was last connected to the Hotspot when it was shut down, it will prioritize starting the Hotspot after reboot, even if the preconfigured Wi-Fi network is available.
- This happens because no specific priority has been set between the two connections.

To address this, we need to run the following two commands in the terminal:

```
sudo nmcli connection modify Hotspot connection.autoconnect-priority 50  
sudo nmcli connection modify preconfigured connection.autoconnect-priority 100
```

By default, both networks have their connection.autoconnect-priority values set to 0. By setting these values, we establish a priority between the two connections. Since the preconfigured network now has a higher priority than the Hotspot, the Raspberry Pi will first check the SSID (Wi-Fi name) and password of the preconfigured network.

If it can connect to the preconfigured network, it will do so. However, if the connection to preconfigured fails, the Raspberry Pi will then activate the Hotspot.

This ensures that the system always prioritizes connecting to the preconfigured Wi-Fi but falls back to the Hotspot if necessary.

Like this:

```
sudo nmcli connection modify Hotspot connection.autoconnect yes  
sudo nmcli connection modify Hotspot connection.autoconnect-priority 50  
sudo nmcli connection modify preconfigured connection.autoconnect-priority
```

Figure 7: Config preconfigured and Hotspot

Now, we've reached an important step. As soon as the Raspberry Pi starts, the appfinal.py file will run automatically. I assume you've already installed the necessary libraries using pip install at the beginning.

Please pay close attention to these steps and follow them carefully.

Firstly, you should enter the following command:

```
sudo nano /etc/systemd/system/run_script.service
```

You will see an empty file. Then, you should add the following lines to this file:

```
[Unit]
Description=Run Python script on startup
After=network.target

[Service]
ExecStart=/usr/bin/python3 /home/pi/your_script.py
WorkingDirectory=/home/pi
User=pi
Group=pi
Restart=always

[Install]
WantedBy=multi-user.target
```

Figure 8: Important Lines

Of course, you change /home/pi/your_script.py and User. I do not know what you write user name but if you upload appfinal.py file to /home/pi folder. Then you write /home/pi/appfinal.py. These lines:

```
[Unit]

Description=Run Python script on startup

After=network.target


[Service]

ExecStart=/usr/bin/python3 /home/pi/appfinal.py
```

WorkingDirectory=/home/pi

User=pi

Group=pi

Restart=always

[Install]

WantedBy=multi-user.target

After save that file, you should enter these commands:

sudo systemctl daemon-reload

sudo systemctl enable run_script.service

sudo systemctl start run_script.service

Command Like this:

```
sudo systemctl daemon-reload
sudo systemctl enable run_script.service
sudo systemctl start run_script.service
```

Figure 9: run_script.service is running

```
pi@raspberrypi:~ $ sudo systemctl status run_script.service
● run_script.service - Run Python script on startup
   Loaded: loaded (/etc/systemd/system/run_script.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-01-06 12:11:38 +03; 13s ago
     Main PID: 637 (python3)
        Tasks: 3 (limit: 1575)
           CPU: 3.418s
    CGroup: /system.slice/run_script.service
            └─637 /usr/bin/python3 /home/pi/appfinal.py
              └─853 /usr/bin/python3 /home/pi/appfinal.py

Jan 06 12:11:38 raspberrypi systemd[1]: Started run_script.service - Run Python script on startup.
Jan 06 12:11:44 raspberrypi python3[637]: * Serving Flask app 'appfinal'
Jan 06 12:11:44 raspberrypi python3[637]: * Debug mode: on
```

Figure 10: Check the run_script.service

If you followed the steps correctly, you should see a screen like the one in Figure 10.

If you want to stop this application from running, you need to enter the following command in the Raspberry Pi terminal:

```
sudo systemctl stop run_script.service
```

However, this will only stop the service temporarily. When you restart the Raspberry Pi, the run_script.service will automatically start again.

If you want to completely prevent it from starting automatically, you need to do the following:

First, stop the service with this command:

```
sudo systemctl stop run_script.service
```

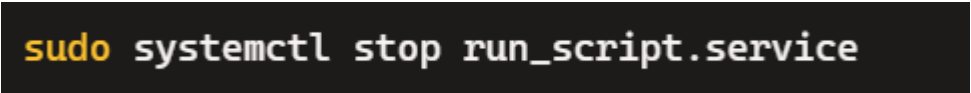
This stops the run_script.service that is currently running on the Raspberry Pi. Then, disable the service with this command:

```
sudo systemctl disable run_script.service
```

By disabling the service, it will no longer start automatically, even after the Raspberry Pi is restarted. If you want to enable and start the service again in the future, you can use these commands:

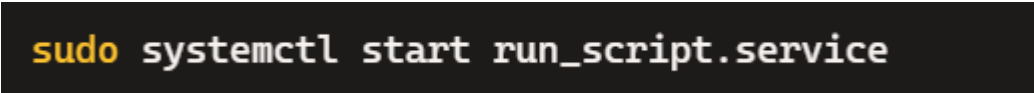
```
sudo systemctl enable run_script.service
```

```
sudo systemctl start run_script.service
```



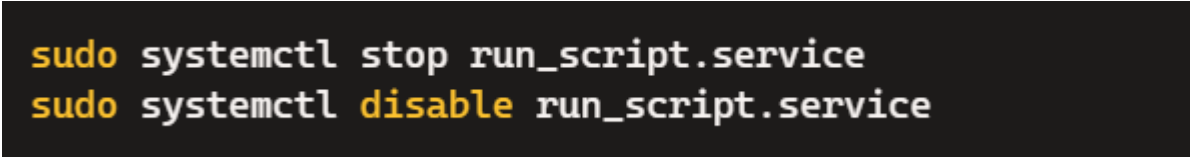
```
sudo systemctl stop run_script.service
```

Figure 11: Stop the run_script.service



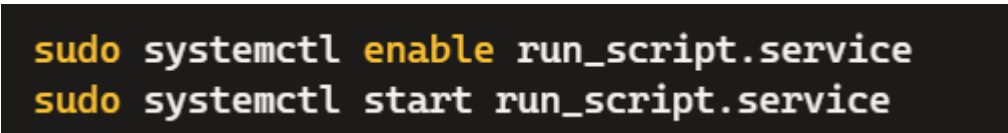
```
sudo systemctl start run_script.service
```

Figure 12: Start the run_script.service



```
sudo systemctl stop run_script.service  
sudo systemctl disable run_script.service
```

Figure 13: Prevent code from autorunning



```
sudo systemctl enable run_script.service  
sudo systemctl start run_script.service
```

Figure 14: Enabling autorun

Now, I will explain the Android Application.

Step 1: Device Scanning Screen

After downloading the Android application, the first screen you will see looks like this:

The purpose of this screen is to scan devices connected to your network. When you click the "Scan the Devices" button, the app will display the subnet IP address of the network you're connected to and the IP addresses of all Linux-based devices on the network. (Since the Raspberry Pi is a Linux-based device, scanning only Linux devices will suffice.)

On this screen, you can select the IP address of your Raspberry Pi and click "Continue".

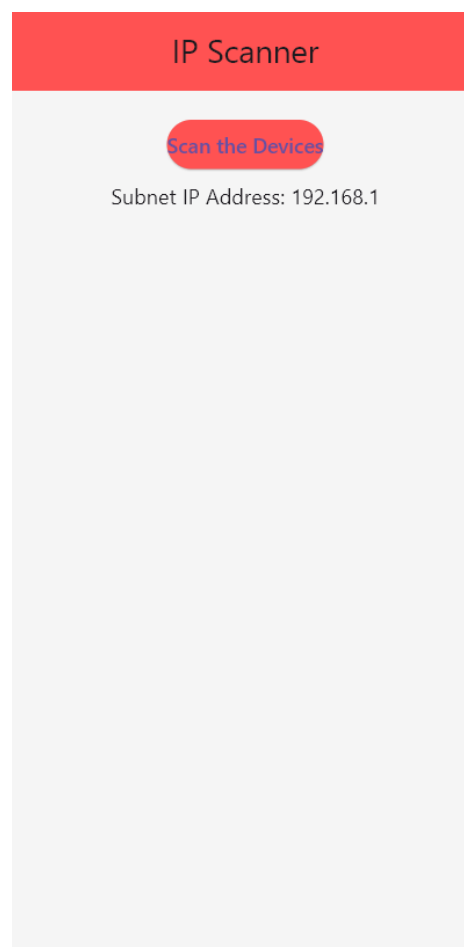


Figure 15: IP Scanner Page

Step 2: Wi-Fi Connection Screen

The second screen you encounter is the Wi-Fi Connection Screen. If your Raspberry Pi is not connected to the internet (for instance, if you're in a new location) and there is a mismatch in the SSID (Wi-Fi name) and password in the preconfigured settings, the Raspberry Pi will switch to Hotspot mode. (This behavior was explained in earlier sections.)

On this screen, you can enter the SSID (Wi-Fi name) and password, and then click "Connect". If the Raspberry Pi is in Hotspot mode, it will try to update the SSID and password in the preconfigured settings and attempt to connect to the specified Wi-Fi network.

If the SSID and password are correct, the Raspberry Pi should successfully connect to the network within 5-10 seconds. Once this process is complete, a hidden "Continue" button will appear.

If you do not need this process, you can switch to the API key screen by pressing the Skip button.

Wi-Fi Connection	Wi-Fi Connection
Enter Wi-Fi Details:	Enter Wi-Fi Details:
Wi-Fi Name (SSID)	Wi-Fi Name (SSID) acrome
Wi-Fi Password	Wi-Fi Password *****
Connect	Connect
Skip	Skip
	Continue

Figure 16: Wi-Fi Connection Page

Step 3: Returning to the IP Scanner

When you click the "Continue" button, you will return to the IP Scanner screen. Keep in mind that the Hotspot will be disabled at this point, so you will need to connect your phone to the same network that your Raspberry Pi is connected to.

If the Raspberry Pi's IP address does not appear immediately, don't worry—it may take a few seconds. You can repeatedly press the "Scan the Devices" button until the Raspberry Pi's IP address shows up. Once you see the IP address, select it and click "Continue".

You will then be taken to the Wi-Fi Connection screen again.

Step 4: Groq API Key Screen

This time, you can press the "Skip" button to move on to the Groq API Key screen. The instructions for obtaining a Groq API Key are included in the README file, so you can refer to that for help.

After entering your API key and clicking "Continue", you will be directed to the main application.

You might wonder, "Do I need to remember my API key every time?" The answer is no. Once you enter the API key and click "Continue", the key will be saved in a .txt file. If your API key is not invalidated, the next time you open the app, you can simply press the "Skip" button on the API Key screen to proceed.

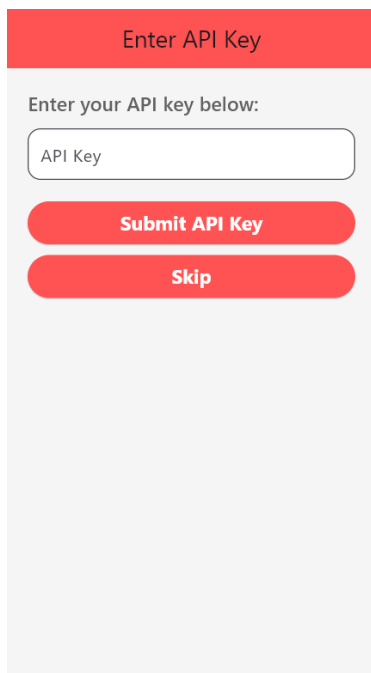


Figure 17: API Key Screen

Main Application

The main application allows you to send commands to an AI system that controls a robot based on the given information.

Additionally, there is a "Start Listening" button that activates the microphone. After you press "Stop Listening", the app will display what you said in a text box.

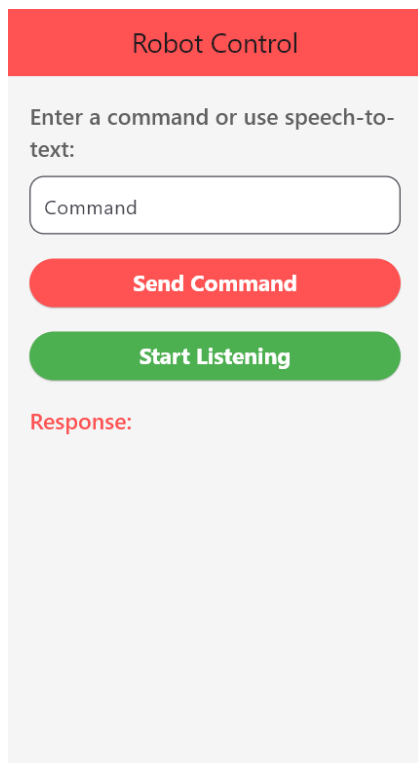
The image shows a mobile application interface titled "Robot Control" in a red header bar. Below the header, the text "Enter a command or use speech-to-text:" is displayed. Underneath this text is a white text input field with the placeholder text "Command". Below the input field are two buttons: a red button labeled "Send Command" and a green button labeled "Start Listening". At the bottom of the screen, the word "Response:" is written in red text, followed by a large, empty light gray rectangular area for displaying the response.

Figure 18: Main Screen

To better understand the function of the application, watch the video from the link below (The video is old version. You will see that there are some innovations in the new version.)

<https://drive.google.com/file/d/1HpLvHrnCPlmwpBCMEQtaDkjluCLsbwnX/view?usp=sharing>